

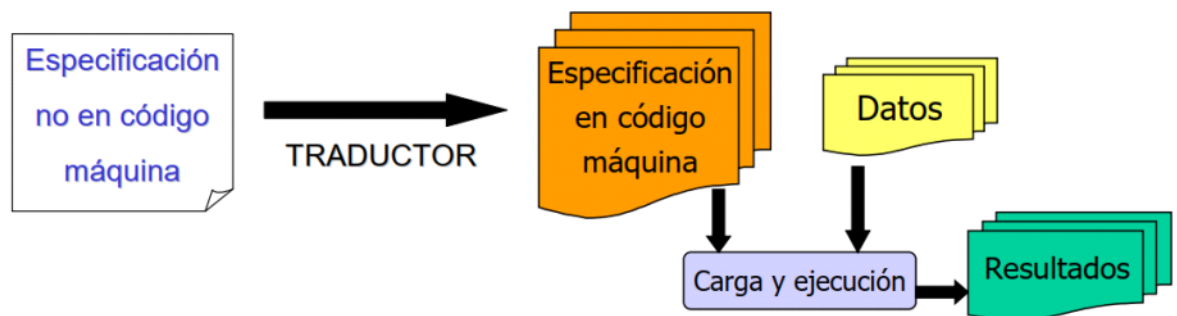
Tema 1 - Procesadores de Lenguajes

Profesor Ramón López-Cózar Delgado, Curso 21-22

1.1 - Conceptos Previos:

Concepto de Traducción

- En el contexto de lenguajes de programación, cuando el lenguaje de especificación de un programa es diferente al lenguaje máquina es necesario traducirlo a lenguaje máquina para que pueda ser ejecutado.



- Un compilador o procesador de lenguaje es un tipo particular de traductor de código fuente a código máquina u objeto.
- Un lenguaje de programación tiene asociada una especie de máquina virtual asociada que entiende ese lenguaje.
- La traducción relaciona dos o más **modelos semánticos** (culturas)
- Un **modelo semántico** está compuesto de:
 - Lenguaje**
 - Conjunto de construcciones simbólicas formadas por secuencias de símbolos de un alfabeto, se utiliza para representar los valores semánticos de un dominio semántico.
 - Un lenguaje puede definirse:
 - Por extensión
 - Por comprensión: Gramática
 - Dominio Semántico**
 - El dominio semántico se compone de valores semánticos elementales (nombres de variables, palabras reservadas, tipos de datos, etc.) y valores estructurados (construcciones *if-else*, bucles, clases, etc.)
 - Interpretación**
 - Sea D un dominio semántico, $L(G)$ el lenguaje definido por la gramática G . Se define la interpretación como una aplicación I :
 - $I: L(G) \rightarrow D$
 - Se denota el modelo semántico (o modelo de la interpretación) como:
 - $M = (L(G), D, I)$

Requisitos para Construir un Traductor

- Dados dos lenguajes $L_1(G_1)$ y $L_2(G_2)$, se necesitan dos modelos semánticos $M_1 = (L_1(G_1), D_1, I_1)$ y $M_2 = (L_2(G_2), D_2, I_2)$ tal que debe cumplirse que:

$$D_1 \subseteq D_2 \text{ y } \forall \alpha \in L_1(G_1), \exists \beta \in L_2(G_2) / I_1(\alpha) = I_2(\beta)$$

- D_1 debe estar incluido en D_2 o ser equivalente, pueden haber más cosas que estén en D_2 , pero no importa en este contexto. Todo lo que se exprese en D_1 , debe poderse expresar en el D_2 y poder

interpretarse.

- "Existe una expresión α en el lenguaje 1, y una expresión β en el lenguaje 2 que pueda ser interpretada de la misma manera"

Esquema de Traducción

- Dados los modelos semánticos $M_1 = (L_1(G_1), D_1, I_1)$ y $M_2 = (L_2(G_2), D_2, I_2)$ se define un *Esquema de Traducción* como una función T :
 - $T: L_1(G_1) \rightarrow L_2(G_2)$
- Donde se verifica que $\forall \alpha \in L_1(G_1), \exists \beta \in L_2(G_2)$ con $T(\alpha) = \beta$ y $I_1(\alpha) = I_2(\beta)$.

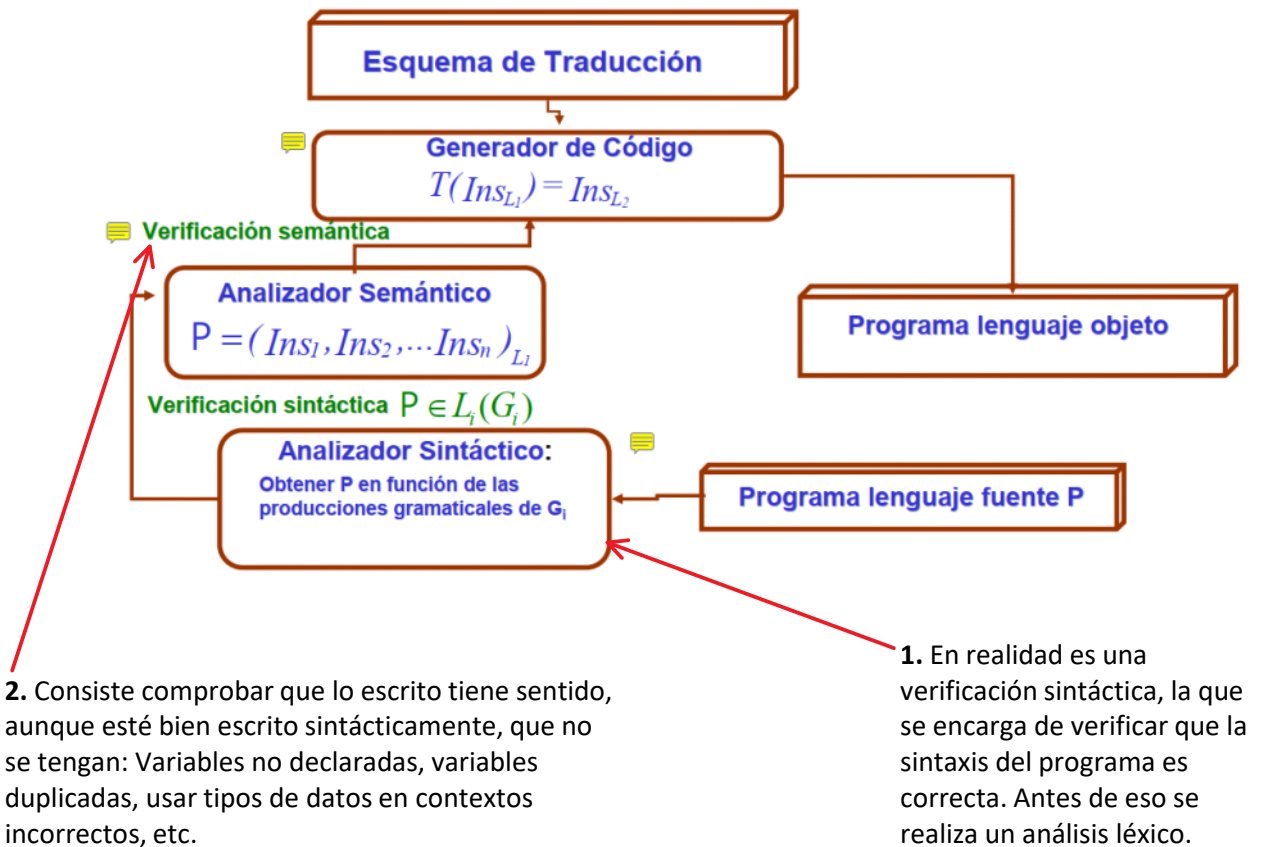
1.2 - Aplicaciones de los Traductores:

Los traductores pueden utilizarse en:

- **Lenguajes de programación**
 - Traducir a código máquina programas escritos en lenguajes de alto nivel (C, Pascal, etc).
 - El dominio semántico (evaluación y control) de los lenguajes de programación coincide con el dominio semántico del lenguaje máquina.
- **Lenguajes orientados al problema**
 - El problema se puede expresar en términos de valores semánticos elementales y estructurados.
 - Se puede definir un lenguaje orientado al problema que permita formular el problema de forma más cercana a la cultura del usuario.
 - Se debe de construir un traductor, ya sea compilador o interprete de este lenguaje a uno de 3ª o 4ª generación.
 - Imaginemos la gestión de un almacén, donde se dispone de *productos*, *clientes*, *proveedores*, así como de las acciones de *compra*, *venta*, *baja* y *pedido*.
 - Se podría diseñar un lenguaje en donde se expresaran las acciones de la forma:
 - *venta* cantidad_1 articulo_1 a cliente_1
 - *si* cantidad_2 < 20 *entonces* *pedido* articulo_1 a proveedor_1
 - *baja* cantidad_1 *existencia actual* cantidad_2
 - Este lenguaje nos permitiría poder expresar nuestro problema a nivel de diseño, permitiendo realizar la fase de codificación de forma automática, mediante un traductor desde el lenguaje diseñado a un lenguaje de alto o bajo nivel.
- **Macroensambladores**
 - Traducen una especificación de un lenguaje de alto nivel a un lenguaje no máquina de 2ª o 3ª generación.
- **Manejadores de bases de datos**
 - Las instrucciones se procesan como las especificaciones para macroensambladores
 - Generan llamadas a funciones de acceso al DBMS como interpretación de las instrucciones de alto nivel.

1.3 - Visión general de un traductor:

- Dados los modelos semánticos $M_1 = (L_1(G_1), D_1, I_1)$ y $M_2 = (L_2(G_2), D_2, I_2)$, su esquema de traducción $T: L_1(G_1) \rightarrow L_2(G_2)$ para traducir un programa $P \in L_1(G_1)$, ¿qué tareas y en que orden deben hacerse?



• Fases

- Análisis
 - Verificación Sintáctica (Análisis Léxico y Análisis Sintáctico)
 - Verificación Semántica (Análisis Semántico)
- Síntesis
 - Generación y optimización del código.

Fase de Análisis: Verificación Sintáctica

- La verificación puede simplificarse teniendo en cuenta lo siguiente:
 - Elegir lenguajes que estén definidos por gramáticas regulares (tipo 3) o gramáticas independientes del contexto (tipo 2).
 - Diseñar lenguajes formados como concatenación de palabras y las palabras formadas como concatenación de símbolos de un alfabeto.
 - Si las frases obedecen a una estructura sintáctica, la verificación sintáctica se puede descomponer en:
 - **Análisis Léxico:** Se identifican las palabras con la misma misión sintáctica.
 - **Análisis Sintáctico:** Se verifica la sintaxis de la secuencia de palabras, se pueden aplicar técnicas predictivas (sin vuelta atrás).

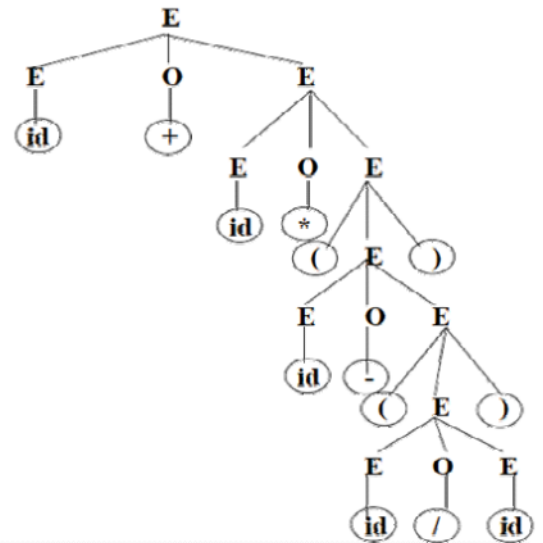
Fase de Análisis: Verificación Sintáctica: Análisis Léxico

- **Componente Léxico (símbolo o token):** Conjunto de palabras que hacen la misma misión sintáctica.
 - Son el mismo token, si al intercambiarlos, el programa sigue siendo sintácticamente correcto.
 - Los tokens son definidos por expresiones regulares.
- **Lexema:** Cada palabra concreta del texto fuente asociada a un token.
- **Patrón:** Regla mediante la cual una secuencia de caracteres del texto fuente es asociada a un token, cada patrón es reconocido por un autómata finito determinista.

- N es el conjunto de símbolos no terminales.
- T es el conjunto de símbolos terminales.
- P es el conjunto de producciones (o reglas) gramaticales.
- S es el símbolo (no terminal) inicial de la gramática.
- Por ejemplo:

$$P = \{ \begin{array}{l} E \rightarrow EOE \\ E \rightarrow (E) \\ E \rightarrow id \\ O \rightarrow + \mid - \mid * \mid / \end{array} \}$$

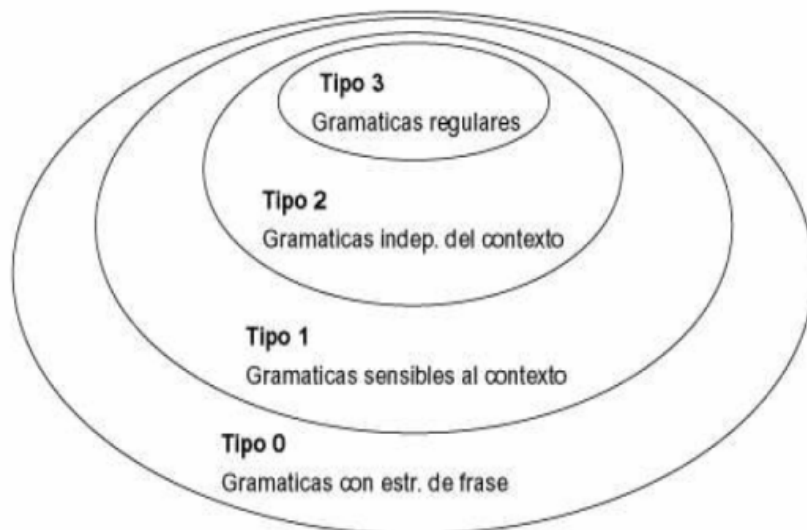
$$\begin{array}{l} N = \{E, O\} \\ T = \{id, (,), +, -, *, /\} \\ S = E \end{array}$$



$$id + id * (id - (id / id))$$

Clasificación de las Gramáticas según Chomsky

Tipo 3	Gramáticas regulares (Secuencias de símbolos: Palabras) $P = \{A \rightarrow aB ; A \rightarrow a / a \in T, B \in (V_N \cup V_T)^*\}$ Son reconocidas por un <i>autómata finito</i>
Tipo 2	Gramáticas independientes del contexto (Secuencias de palabras) $P = \{A \rightarrow \alpha / A \in V_N ; \alpha \in (N \cup T)^*\}$ Son reconocidas por un <i>autómata con pila (push-down)</i>
Tipo 1	Gramáticas sensibles al contexto $P = \{\alpha A \beta \rightarrow \alpha \gamma \beta / A \in V_N ; \gamma \in (V_N \cup V_T)^+ ; \alpha, \beta \in (V_N \cup V_T)^*\}$ Son reconocidas por un <i>autómata lineal limitado</i>
Tipo 0	Gramáticas con estructura de frase $P = \{\alpha \rightarrow \beta / \alpha \in (V_N \cup V_T)^+ ; \beta \in (V_N \cup V_T)^*\}$ Son reconocidas por una <i>máquina de Turing</i>



- Para asegurarse que una cadena de símbolos pertenece a un lenguaje, es necesario aplicar todas las combinaciones posibles sobre las producciones de la gramática y comprobar que si puede generarse, lo cual no es siempre posible o recomendable: la fuerza bruta no es la solución
- Suponer que esta es la entrada:

$$id + id * (id - (id / id))$$

y supongamos la gramática:

$$E \rightarrow EOE, E \rightarrow id, O \rightarrow + \mid -$$

La aplicación de producciones nunca terminaría.

- **¿Cómo se soluciona esto?** Con análisis sintáctico predictivo.
- Se utilizan abstracciones léxicas también para simplificar la gramática original en otra gramática denominada abstracta.
- La estrategia de análisis puede ser ascendente o descendente.

Fase de Análisis: Verificación Sintáctica: Análisis Sintáctico Predictivo

- Ante un símbolo de entrada, se permite decidir qué hacer para proseguir el análisis y obtener una cadena del lenguaje, sin tener que generar todas las posibles cadenas.
- Es válido si la gramática no es ambigua.
- Existen técnicas de análisis predictivo dependiendo de la estrategia de análisis
 - Descendiente: LL(1)
 - Ascendente:
 - Precedencia
 - LR
 - SLR
 - LALR
- La herramienta para el análisis sintáctico es YACC

Fase de Análisis: Verificación Semántica: Análisis Semántico

- Desde el árbol sintáctico (la secuencia de producciones aplicadas en el análisis) podemos identificar las secuencias de componentes léxicos con significado conjunto.
- Ya que se parte de una secuencia de componentes léxicos correcta sintácticamente, podemos analizar el valor semántico estructurado asociado con cada subsecuencia de componentes léxicos junto con sus argumentos.

- En el ejemplo anterior, tenemos dos símbolos no terminales de la gramática que identifica secuencias de entrada con significado conjunto:

$$\begin{array}{ll} C \rightarrow \text{if } E \text{ then } S & \text{-----} \rightarrow C \rightarrow \text{IF } E \text{ THEN } S \\ A \rightarrow id := E & \text{-----} \rightarrow A \rightarrow \text{ID ASIGN } E \end{array}$$

- Al aplicar la producción $A \rightarrow \text{ID ASIGN } E$, debemos comprobar que el lexema asociado con **ID** debe ser del mismo tipo que el de la expresión **E**. Esto nos facilita completar la información acerca de los lexemas. Nos permite completar la información de la tabla de símbolos.
- Verificaciones semánticas.

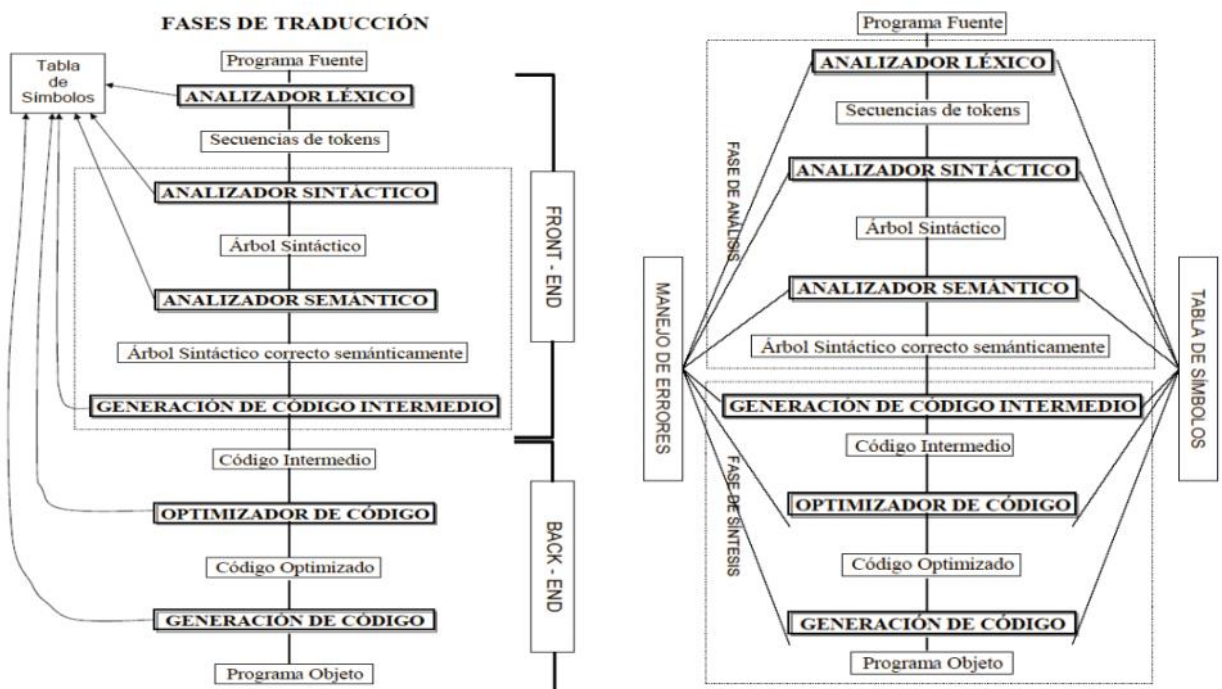
Fase de Síntesis: Generación de código

- Consiste en aplicar de forma particularizada el Esquema de Traducción para cada secuencia de lexemas con sentido propio
- En esta etapa se obtiene código intermedio.
- Un aspecto importante en los traductores de lenguajes de programación consiste en que deben generar códigos de una eficiencia comparable al generado directamente en lenguaje máquina.
- Esta cualidad deseable de los traductores ha dado lugar al desarrollo de técnicas de optimización del código

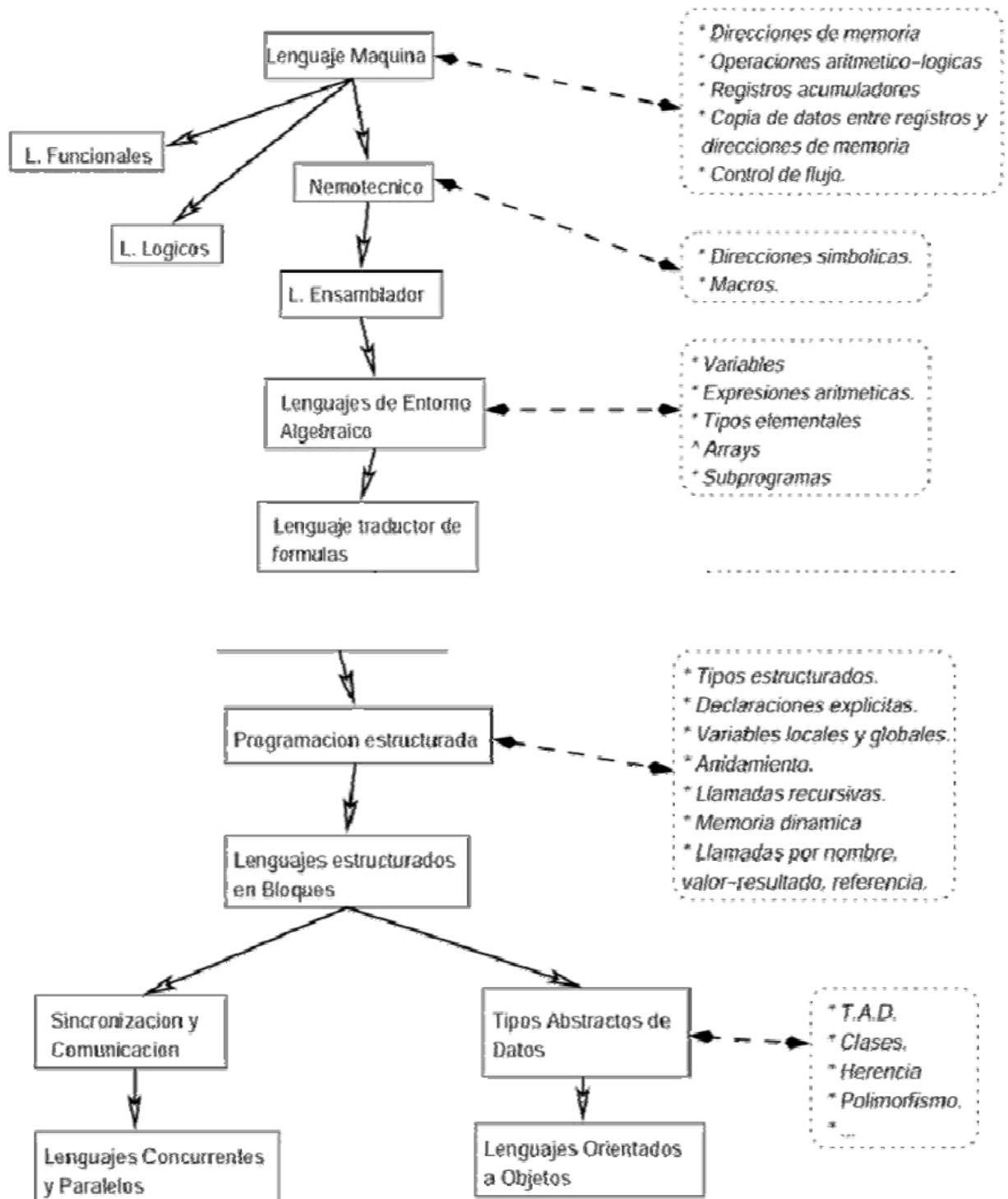
Fase de Síntesis: Optimización del código

- Toda secuencia de programa obtenido en código máquina u objeto se caracteriza por la presencia de:
 - Instrucciones de asignación
 - Instrucciones de evaluación
 - Instrucciones de control
- En ocasiones es posible reorganizar el código para reducir el número de instrucciones

Modelo general de un traductor



1.4 Tipos y evolución de los traductores:

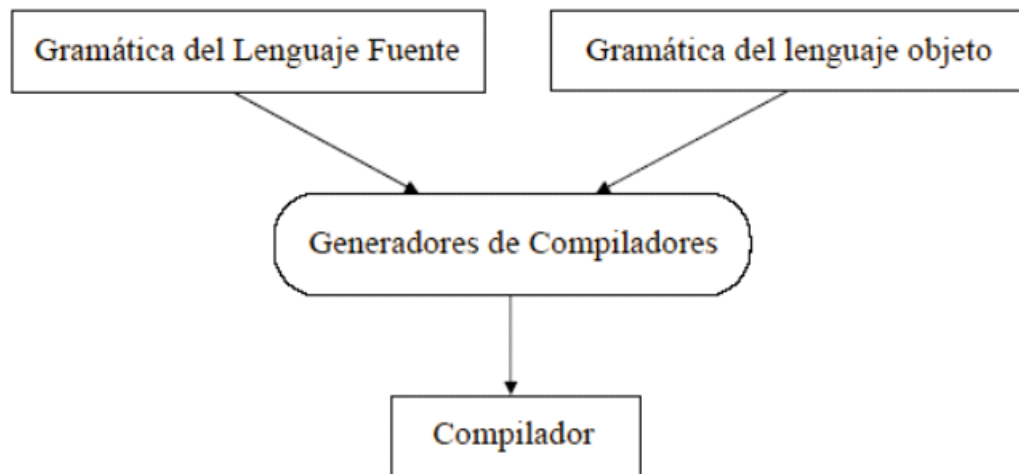


- Los traductores han ido evolucionando junto con los propios lenguajes de programación.

Evolución de los traductores

- Década de los 50
 1. Se examinaba el texto de entrada.
 2. Se construía la tabla de símbolos y de direcciones.
 3. Se traducían las direcciones simbólicas a direcciones en código.
 4. Se generaba el código
- Década de los 60
 1. Se hacía el análisis léxico
 - Gramáticas regulares o libres de contexto
 2. Luego se hacía en análisis semántico
 3. Se generaba el código

- **Generadores de compiladores:** Traductores dirigidos por sintaxis
 - Dada las gramáticas de dos lenguajes, un generador de compilador genera un compilador entre ellos.

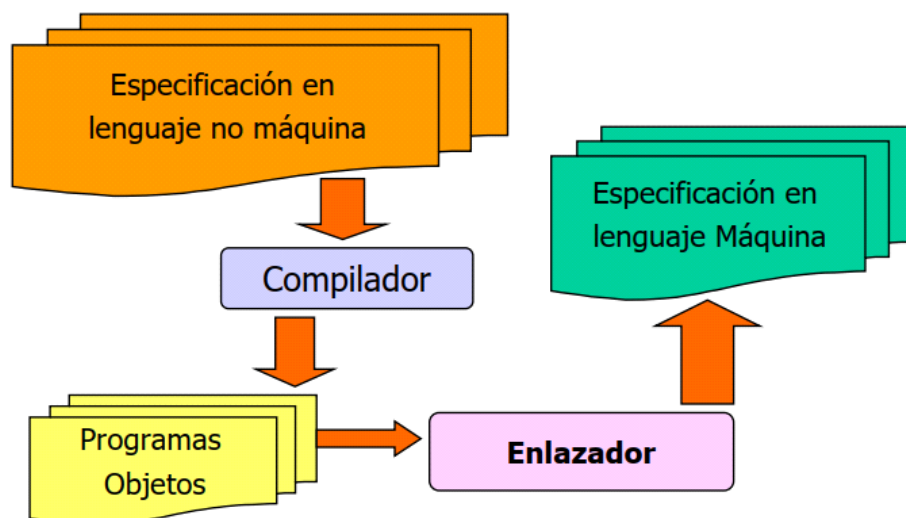


Incidencia de conceptos de lenguajes de programación en traductores

- Lenguajes de entorno algebraico
 - Variables: Usar un mapa de memoria estática.
 - Transferencia de argumentos por referencia: Usar tabla de símbolos.
- Lenguajes de programación estructurada.
 - Tipos estructurados.
 - Declaraciones explícitas.
 - Variables locales y globales: Usar tabla de símbolos por bloques.
 - Anidamiento: Usar tabla de símbolos por bloques.
 - Dimensiones dinámicas: Gestión dinámica de memoria.
 - Subprogramas recursivos: Gestión dinámica de memoria.
 - Transferencia de argumentos: Usar tabla de símbolos por bloques.

Compiladores

- Un compilador obtiene una especificación en lenguaje máquina equivalente

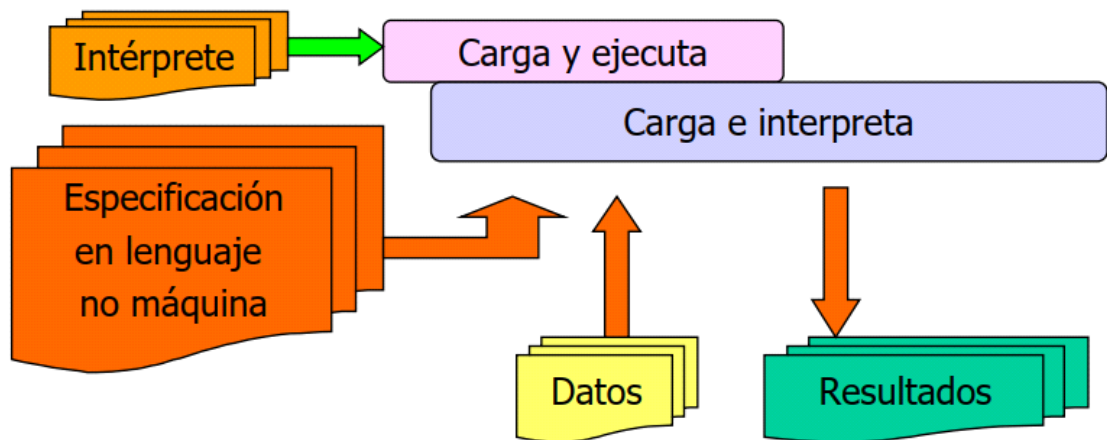


- **Compilador:** Traduce la especificación de entrada a lenguaje máquina incompleto y con instrucciones máquina incompletas
- **Enlazador:** Enlaza los programas objetos y completa las instrucciones máquina incompletas, generando

un ejecutable.

Intérprete

- Un intérprete carga la especificación de un programa en lenguaje no máquina y la interpreta y ejecuta, instrucción a instrucción.



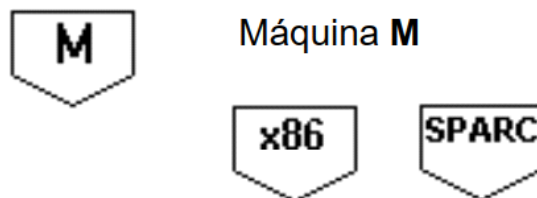
1.5 Arquitectura de los Procesadores de Lenguajes:

- Los traductores son programas que manipulan otros programas.
- Hay tres lenguajes involucrados:
 - Lenguaje Fuente: S (Source)
 - Lenguaje Objeto: T (Target)
 - Lenguaje de implementación de traductor: L (Language)

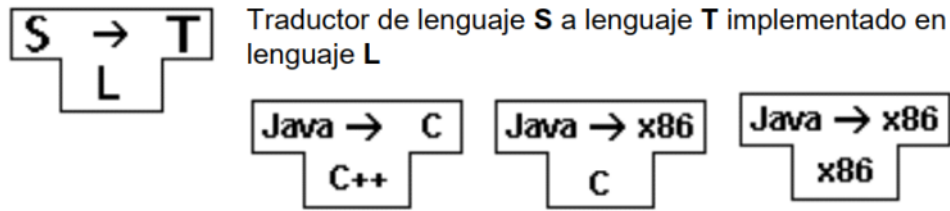
- **Representación “TOMBSTONE” (“Lápida sepulcral”, D. A. Watt, 1993)**
 - Diagrama para un programa



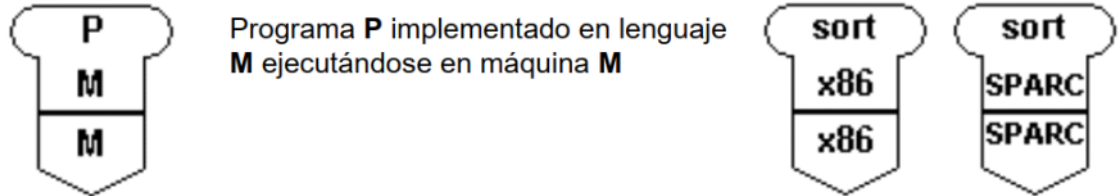
- Diagrama para una máquina



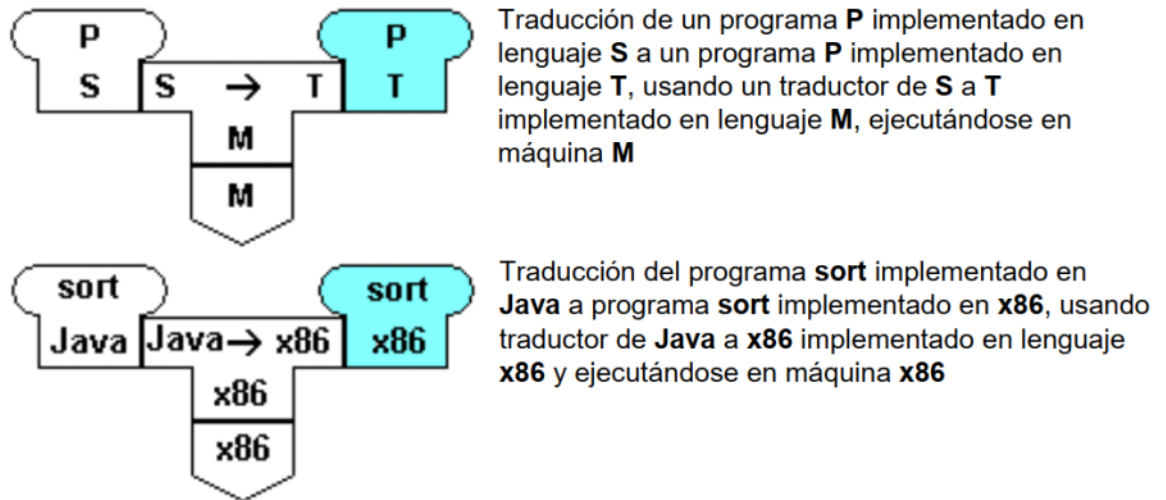
- Representación “TOMBSTONE” (“Lápida sepulcral”, D. A. Watt, 1993)
 - Diagrama para un traductor



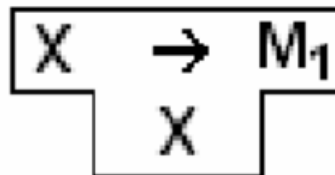
- Diagrama para un programa en ejecución



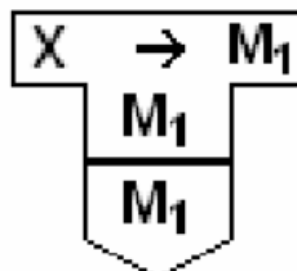
- Representación “TOMBSTONE” (“Lápida sepulcral”, D. A. Watt, 1993)
 - Diagrama para un traductor en ejecución



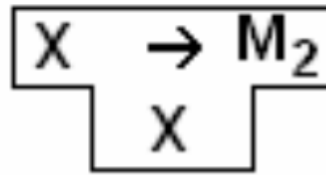
- ¿Cómo se portaría un traductor de una máquina M_1 a una máquina M_2 ?
 - Se tiene el compilador existente que se quiere portar:



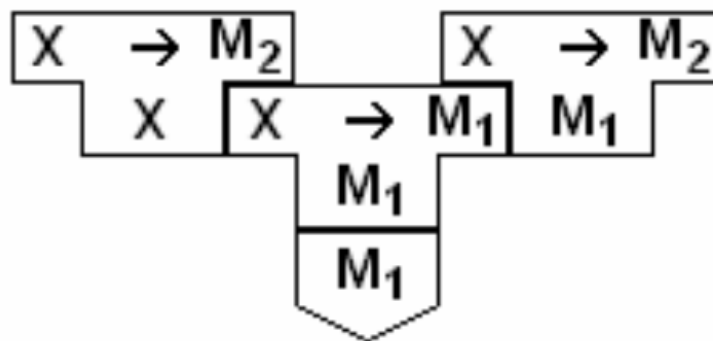
- Se dispone del código de un compilador de lenguaje X que se ejecuta en la máquina M_1 y produce código para máquina M_1 .



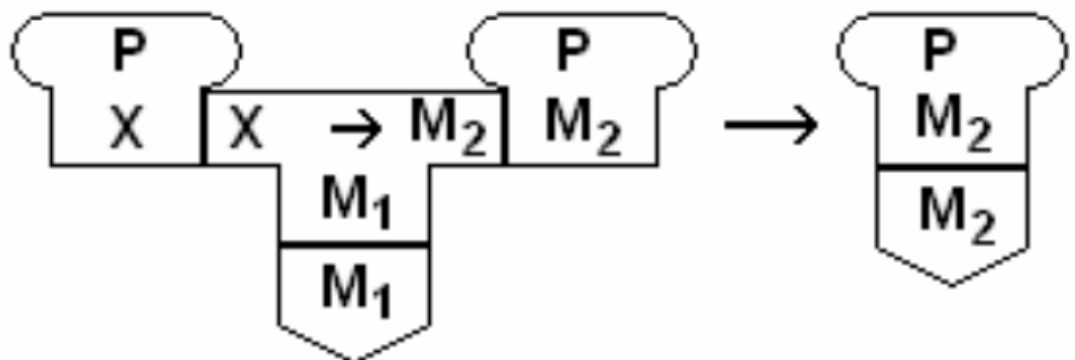
- Se puede utilizar algo llamado **Half Bootstrap**.
 - **Paso 1:** Reescribir la parte de generación de código del compilador existente para que produzca código para la nueva máquina M_2



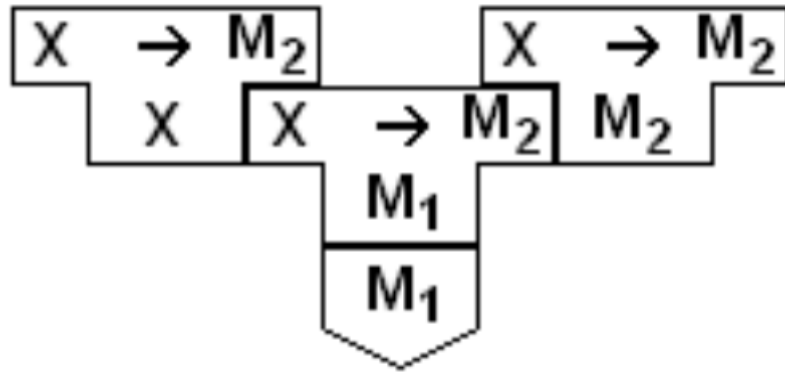
- **Paso 2:** Compilar en la primera máquina el compilador modificado



- **Resultado:** Compilador que se ejecuta en la máquina M_1 y produce código para la máquina M_2 . Esto es un compilador cruzado.
- Se pueden ejecutar en la máquina M_2 programas escritos en X compilados por la máquina M_1 .



- El compilador cruzado puede ser utilizado para compilarse así mismo, produciendo un compilador que se puede transferir a la máquina M_2 .



- **Bootstrapping:** Traducir un lenguaje complejo usando un lenguaje simple.
 - Como un compilador de C++ escrito en ensamblador